IOWA STATE UNIVERSITY

Aircraft Data Visualization

Review of Version One / HCI584 / Andy Walker

Project Introduction

In the past, radar research has been limited to large companies and those with exceptional budgets. However, Amateur Radio enthusiasts are increasingly able to buy and build capable microwave systems for communications, sharing of data, and sensing of obstacles. These capabilities are primed to push amateur electronics into the realm of real microwave sensors like radars. Radars use radio signals to detect and track 'targets' in their field of view, and larger targets make for easier, more certain detections. Commercial aircraft provide excellent targets of opportunity for such a system. They are physically and electrically large, they move at high speed and broadcast their position regularly via radio links that can be used to provide a source of truth for the hobbyist measurement. This enables educational radar training by providing a test bed for unclassified, amateur radio compatible radar. This project captures the broadcast position data and maps it relative to the sensor location to provide 'targets of opportunity' to the sensor.

Version 1 Review

Functions Implemented:

def calculate_slant_range(radar, plane): Calculates the azimuth and elevation angles from the sensor location to the aircraft as well as a slant range. The slant range is the range that the sensor would need to detect and track over.

def read_gps_coordinates(serial_port='COM5', baud_rate=4800, timeout=1, max_attempts=50): Reads the location of the sensor from a GPS receiver on COM5 (default). This output will be refined with information from the Iowa Real Time Kinematic Network in future work.

def calculate_radar_range(pt_watts=250, gain_db=26, num_pulses=1000, freq_hz=2.45e9, rcs_sqm=1.0, s_min_watts=1e-13, loss_db=0): Calculate the effective range of the sensor given 1000 coherenet pulses. This is the equation used to generate the range 'stop-light' rings shown on the map.

def call_api(latitude, longitude, altitude, limit_range="27", units="M"): Collects the data from the ADS-B API.

def plot_map (latitude, longitude, range_10, range_20, range_30, my_map): Initializes 'my_map' and plots the range rings that show what aircraft are within detection range of the sensor.

def plot_plane (coord1, coord2, my_map, description, col = "blue"): Plots the locations of the planes. One of the next steps is to add unobtrusive persistence to this plotting.

def get_masking(coord1, coord2, num_segments): Calculates the elevation at each point along a line projected on the ground between the sensor and the aircraft. If any of the elevations are higher than the elevation of the line of sight between the sensor and the aircraft, the line of sight is marked 'masked' and the aircraft is plotted in red to show that it is probably not visible to the sensor.

def filter_list(r): Consumes the JSON file 'r' and filters out the information from the ADS-B return that is used in the subsequent processing. Altitude, latitude, and longitude are copied from input to output. Heading is available but not used; instead the tracking algorithm should use a 'plausibility circle' to determine which aircraft past state corresponds to which aircraft current state.

def set_location(Port="COM5"): Has become the single function for defining the location of the center node. Accommodates GPS coordinates or variable defaults. Applies to AC_Range.py, app.py, and streamlit_app.py

def rotate_icon(angle): Rotates a PNG icon to a an angle specified by the caller and saves (overwrites) the newly rotated image for use as an icon on a map.

def plot_vector(coord1, coord2, my_map, col = "blue"): Takes as input the location of an aircraft along with it's last location to generate a 'trail of ants' vector that points in the direction of travel. WIP.

Record a video of your screen with voice-over:

See Canvas Studio video.

Issues and Solutions:
Talk about issues you encountered (i.e. things that you thought would work but, as it turned out, didn't) and how you solved them (if you did).
For all issues that you haven't solved yet, outline how you propose to deal with them (Do something totally different? Spend more time and hope it can still be solved?)

Where is help needed? I think I'm nearing the point where I'd like an hour of your time to ensure I'm doing my geography math effectively. I think there's a lot I can get away with since I'm not interested in any measurements beyond the possible range of my sensor, nominally 100 kilometers. I am testing the terrain masking in several locations around the world but given the small sensor range, most locations are pretty flat relative to the typical altitudes of aircraft.
After that I'd like help to optimize the delivery, i.e. refresh flickering. A good example of the ideal is at https://www.flightradar24.com/. See the current-state videos for examples of refresh flickering.

Milestones:

1) Multitarget Tracker: Implement the multitarget tracker using current state and previous state(s) to calculate a motion vector. Apply that motion vector to aircraft icons that reflect the heading of the aircraft without resorting to ADS-B data.

2) Graphical User Interface Smoothing: Modify execution order to avoid redrawing the map / html file on each update to make the user interface smoother. Ideally the aircraft icons would relocate on each iteration but leave behind a trail of dots showing 10 previous locations.
3) GUI User Inputs: Make the GUI User Inputs live such that changing the latitude and longitude in the GUI changes the location of the sensor and all downstream calculations including API pulls.
4) GUI Formatting: Generate a user interface tuned for a small screen (e.g. cell phone) with simple, touch-screen compatible controls.

Self-reflection:
Are you satisfied with your progress so far? – I really am. I was hoping I'd be able to make a web application, and it looks like I will. Beyond that, I'm pleased with the focus on GIT and working with someone else. I don't expect to ever be a 'real developer' but if can add value to projects at work, I'll feel like one.

Do you think you can finish the project within the next weeks? Yes, I think I can. I have met all of the Phase 1 Requirements and Objectives except Requirement 7 and Objective 3 (Tracking targets) and those are up next. I have done testing of the different functionalities and gotten consistent results that match expectations and I'm actively working to handle all exceptions found in testing to make the application robust.

Expectations vs. reality: what was easier, what was more difficult than you expected? It's been a bit of struggle keeping track of where default values are kept. I have different ways to call the worker functions and each of those ways has their own defaults. I'm pushing all those defaults into one place now so I can modify them just once.

Any light bulb moments? In actually working on your project, did you discover something that you had not expected? I didn't expect that working with external, online data sources would be as easy as it turned out to be. I did have to work to filter the JSON return data by hand but once I got a handle on the data format (a dictionary of lists of dictionaries) I was able to extract what I wanted without too much trouble.

Project Specification Updates: I think the spec is in pretty good shape as-is. The only change I made was to eliminate the plot of the line-of-sight vectors (from sensor to aircraft) and replace those with icons of each aircraft. The line-of-sight vectors made the plots too busy and didn't really give as much information as I hoped. I have not removed the folium plots from the code – I may just make them faint, dotted, etc.
I also think that the example dashboard is too busy and needs to be reduced. I can imagine a clean way to put the controls on one phone-sized screen and then use swipes to control the different nodes. That's a 'nice to have'.

Follow-On work

Enable Real-Time Kinematic (RTK) data corrections for ultra-accurate central location measurement

The Iowa Real Time Network (https://iowadot.gov/consultants-contractors/design/iowa-real-time-network) is Iowa's global navigation satellite system (GNSS) reference station network. The goal of the network is to provide a series of fixed-site (surveyed) GNSS (e.g. GPS) receivers that publish the measurements that those receivers make. Since the receivers know where they are with high precision and don't move, the errors they measure can be broadcast on the internet and used to correct for atmospheric disturbances to GPS receivers that are mobile. Specialized receivers called Real-Time Kinematic (RTK) receivers apply the corrections broadcast by IaRTN to improve GPS receiver accuracy to within 2 inches.

Putting 2 of these receivers on a short boom (roughly 36") attached to the radar antenna enables 2 separate high precision measurements that can be used to calculate the azimuth and elevation pointing angles of the antenna. We can then know the precise location of the antenna on the surface of the earth as well as which direction it's pointed and if there is any tilt in the boom relative to the surface of the earth. The state of Iowa provides open geospatial APIs and datasets to develop custom software, building scripts, or mapping applications:

Conclusion

This specification captures the requirements and objectives of the first phase of the algorithm and graphical user interface (GUI) development. Remaining work is captured in the Follow-On section.

CONTACT INFORMATION

Any questions can be directed to the author, Andy Walker, at the following:

- andyw@iastate.edu
- andywalker2001@hotmail.com
- 319-538-4032

After working through the spec, what was the biggest or most unexpected change you had to make from your sketch? – I hadn't considered the tracking use case and while a linear extrapolation is useful for commercial airliners, smaller hobby planes will have more erratic flight paths and a more involved tracking algorithm may be required.
How confident do you feel that you can implement the spec as it's written right now? – I think I have the bones of this now, and keeping a log of the locations over time to make a state vector for eac aircraft seems like the logical next step.
What is the biggest potential problem that you NEED to solve (or you'll fail)? – I think the biggest problem I've seen so far is the flickering in the GUI. I'll need help to make a smoothly updating web app.
What parts are you least familiar with and might need my help? – The GUI and controls pieces, although Python libraries sure seem to help…